

```

from flask import Flask, request, jsonify
from flask_cors import CORS

app = Flask(__name__)
CORS(app)  # Allows the HTML website to call this API

# --- Lookup Tables ---

# U-values (W/m²K) by insulation quality
# Represents overall heat transfer coefficient for walls/roof combined
U_VALUES = {
    "poor": 1.8,
    "average": 1.0,
    "good": 0.4
}

# Window heat loss multiplier (fraction of floor area that is window)
WINDOW_AREA_FRACTION = {
    "low": 0.10,
    "medium": 0.20,
    "high": 0.35
}

# Window U-value (W/m²K) - windows lose heat faster than walls
WINDOW_U_VALUE = 3.0

# Temperature difference (°C) between inside (21°C) and outside by climate zone
DELTA_T = {
    "cold": 30,  # e.g. -9°C outside
    "temperate": 15,  # e.g. 6°C outside
    "warm": 8  # e.g. 13°C outside
}

# Ceiling height assumption per building type (metres)
CEILING_HEIGHT = {
    "house": 2.5,
    "apartment": 2.7,
    "office": 3.0,
    "warehouse": 6.0
}

# Worst-case baseline U-value (poor insulation, high windows, cold climate)
# Used to calculate efficiency percentage
BASELINE_U = U_VALUES["poor"]

```

```
BASELINE_WINDOW_FRACTION = WINDOW_AREA_FRACTION["high"]
```

```
BASELINE_DELTA_T = DELTA_T["cold"]
```

```
def estimate_surface_area(floor_area, num_floors, building_type):
```

```
    """
```

```
    Estimate total heat-loss surface area (walls + roof).
```

```
    Floor area per floor = total floor area / num_floors.
```

```
    Wall area = perimeter * height * num_floors (perimeter estimated from square
```

```
    Roof area = floor area of top floor only.
```

```
    """
```

```
    floor_area_per_level = floor_area / num_floors
```

```
    side_length = floor_area_per_level ** 0.5 # Assume square floor plan
```

```
    perimeter = 4 * side_length
```

```
    ceiling_h = CEILING_HEIGHT[building_type]
```

```
    wall_area = perimeter * ceiling_h * num_floors
```

```
    roof_area = floor_area_per_level # Only top floor has a roof
```

```
    return wall_area, roof_area
```

```
def calculate_heat_loss(floor_area, num_floors, building_type,
```

```
                        insulation, windows, climate):
```

```
    """
```

```
     $Q = U * A * \Delta T$  (in Watts, converted to kW)
```

```
    Wall and roof loss + window loss calculated separately.
```

```
    """
```

```
    wall_area, roof_area = estimate_surface_area(floor_area, num_floors, building
```

```
    total_opaque_area = wall_area + roof_area
```

```
    # Window area estimated from floor area
```

```
    window_area = floor_area * WINDOW_AREA_FRACTION[windows]
```

```
    # Subtract window area from opaque area to avoid double-counting
```

```
    opaque_area = max(total_opaque_area - window_area, 0)
```

```
    u = U_VALUES[insulation]
```

```
    dt = DELTA_T[climate]
```

```
    wall_loss = u * opaque_area * dt
```

```
    window_loss = WINDOW_U_VALUE * window_area * dt
```

```
    total_loss_watts = wall_loss + window_loss
```

```
    total_loss_kw = round(total_loss_watts / 1000, 2)
```

```
    return total_loss_kw
```

```
def calculate_baseline_loss(floor_area, num_floors, building_type):
```

```

"""
Worst-case heat loss used as the 0% efficiency reference point.
"""

wall_area, roof_area = estimate_surface_area(floor_area, num_floors, building
total_opaque_area = wall_area + roof_area
window_area = floor_area * BASELINE_WINDOW_FRACTION
opaque_area = max(total_opaque_area - window_area, 0)

wall_loss = BASELINE_U * opaque_area * BASELINE_DELTA_T
window_loss = WINDOW_U_VALUE * window_area * BASELINE_DELTA_T
return (wall_loss + window_loss) / 1000

def calculate_efficiency(actual_loss_kw, baseline_loss_kw):
    """
    Efficiency = how much better than worst-case, as a percentage.
    100% = no heat loss (perfect). 0% = worst case.
    """
    if baseline_loss_kw == 0:
        return 100.0
    efficiency = (1 - actual_loss_kw / baseline_loss_kw) * 100
    return round(max(0, min(100, efficiency)), 1)

# --- API Route ---

@app.route("/calculate", methods=["POST"])
def calculate():
    data = request.get_json()

    # --- Validate inputs ---
    required_fields = ["building_type", "floor_area", "num_floors",
                       "insulation", "windows", "climate"]
    for field in required_fields:
        if field not in data:
            return jsonify({"error": f"Missing field: {field}"}), 400

    building_type = data["building_type"].lower()
    floor_area = data["floor_area"]
    num_floors = data["num_floors"]
    insulation = data["insulation"].lower()
    windows = data["windows"].lower()
    climate = data["climate"].lower()

    # Type checks
    if building_type not in CEILING_HEIGHT:
        return jsonify({"error": "Invalid building_type. Choose: house, apartment

```

```

if insulation not in U_VALUES:
    return jsonify({"error": "Invalid insulation. Choose: poor, average, good"})
if windows not in WINDOW_AREA_FRACTION:
    return jsonify({"error": "Invalid windows. Choose: low, medium, high"}),
if climate not in DELTA_T:
    return jsonify({"error": "Invalid climate. Choose: cold, temperate, warm"})
if not isinstance(floor_area, (int, float)) or floor_area <= 0:
    return jsonify({"error": "floor_area must be a positive number"}), 400
if not isinstance(num_floors, int) or num_floors < 1:
    return jsonify({"error": "num_floors must be a positive integer"}), 400

# --- Calculate ---
energy_loss_kw = calculate_heat_loss(
    floor_area, num_floors, building_type, insulation, windows, climate
)
baseline_kw = calculate_baseline_loss(floor_area, num_floors, building_type)
efficiency = calculate_efficiency(energy_loss_kw, baseline_kw)

return jsonify({
    "energy_loss_kw": energy_loss_kw,
    "thermal_efficiency": efficiency
})

@app.route("/health", methods=["GET"])
def health():
    return jsonify({"status": "ok"})

if __name__ == "__main__":
    app.run(debug=True, port=5000)

```